

Prompt Injection Detection Using Lexical, Linguistic, and Semantic Features: A Comparative and Interpretable Machine Learning Study

Ishika Gupta
B.Tech
Mathematics and Computing
IGDTUW
Kashmere Gate, Delhi
ishika025btmac25@igdtuw.ac.in

Khanak Agrawal
B.Tech
Mathematics and Computing
IGDTUW
Kashmere Gate, Delhi
khanak034btmac25@igdtuw.ac.in

Abstract— Prompt injection attacks are a major source of concern in deployment of Large Language Models (LLMs) based applications. Our study proposes a hybrid machine learning architecture integrating TF-IDF features, linguistic keyword-based features, and semantic sentence embeddings generated using the all-MiniLM-L6-v2 model, to distinguish malicious from benign prompts. We tested six different Machine Learning Classifiers including Naive Bayes, Decision Tree, Logistic Regression, Linear SVM, Random Forest and XGBoost using hyperparameter tuning and cross-validation. Linear Support Vector Machine achieved the best performance with a 5-fold cross-validated F1-score of 0.932 (± 0.019) and test-set accuracy of 91.7%. We further conducted an Explainability Analysis in which we identified the linguistic patterns and keywords that contributed to the detection of malicious prompts. From our research we found out that by combining lexical, linguistic and semantic features we made our model precise, comprehensible and computationally efficient against prompt injection attacks.

Keywords—*Prompt injection detection; Large language models; Feature engineering; TF-IDF; Sentence embeddings; Machine learning classifiers; Ablation study; Explainability*

1. INTRODUCTION

Prompt injection attacks are types of attacks that target LLM based applications in which malicious prompts are used to override intended model behaviour and force unsafe outputs. Since these attacks use natural language rather than a fixed syntax, they're much harder to catch with traditional filtering. This has prompted extensive research in this area.

The two broad ways in which researchers have tried to solve this problem are:

The first method includes surface level detection like by identifying specific phrases, unusual punctuation, capitalization etc. This approach is easy as well as cheap but can easily be beaten by rephrasing an attack.

The second method includes encoding prompts as dense vectors and then allowing a classifier to find underlying patterns and structures. It is more resilient to rephrasing but it

can't explain why it flagged something and can act as a black box once trained.

Most research works lean firmly into one of these methods. Also, in many studies, the main target is to provide an accuracy number to their model. They do not conduct proper ablation of features and the models are not optimized (hyperparameter tuning for example) to their full potential.

Our project is a combination of these two methods. We build a feature block that combines lexical statistics (TF-IDF), keyword and linguistic signals and sentence embeddings. The former for surface signal and latter for semantic signal. Rather than only reporting how well the combined model performs, we test each of three features separately (ablation) to see how much each of them contributes. Further, six traditional ML classifiers are tuned and compared on this feature block.

Our three main research questions include:

- **RQ1:** Does prompt length alone separate benign prompts from malicious ones?
- **RQ2:** How much does each feature block (TF-IDF, keyword/linguistic and embeddings) contribute individually and after being combined?
- **RQ3:** Which traditional ML classifier performs best on the combined feature space after proper tuning?

Contributions: (1) a feature engineering block combining lexical statistics (TF-IDF), hand-crafted keyword and linguistic signals and sentence embeddings; (2) a controlled ablation isolating each block's individual contribution; (3) systematic hyperparameter tuning across six traditional ML classifiers under equal training conditions; and (4) an interpretability analysis identifying which keyword and linguistic features most strongly drive malicious versus benign predictions.

2. BACKGROUND AND RELATED WORK

2.1 Traditional ML for Prompt Injection Detection

- Ref. [1] compared traditional ML, fine-tuned transformers, specialized detectors, and LLM-based classifiers on a shared dataset. Fine-tuned transformers were the best but traditional ML was also not much behind, achieving a 96.69% F1 score at a fraction of the computational cost.
- Within traditional ML, feature choice matters as much as classifier choice. Ref. [2] trained LSTM, feed-forward, Random Forest, and Naïve Bayes classifiers on the HackAPrompt dataset. Random Forest came out the strongest. But since Random Forest was trained on considerably more data than the other models, it's unclear whether the same ranking would hold under equal conditions.
- Ref. [6] took a hybrid approach for multilingual (Hindi/Hinglish) detection. It combined TF-IDF-based traditional ML with transformer and rule-based components. The accuracy achieved was very high (99.70%). The study however doesn't distinguish how accuracy was affected when only traditional ML was used and when this new hybrid approach was applied.
- Neither study performs systematic hyperparameter tuning, so we can't know for sure if each model's true potential was reached and if it reflected in the results.

2.2 Embedding-Based Detection

Using OpenAI's text-embedding-3-small, Ref. [3] converted a large selection of prompts into embeddings. Random Forest Classifiers which were trained on these embeddings showed strong detection. This suggests that dense semantic vectors capture attack-relevant patterns that keyword rules would miss. But here no explanation is given as to why the model gave a certain decision, lacking feature-level interpretability.

2.3 Transformer and LLM-Judge Approaches

Ref. [4] built a dedicated detection system (PromptShield) using a fine tuned transformer. It achieved a very strong AUC(0.998) but at the cost of heavier training and deployment requirements compared to a traditional classifier. Ref. [5] combined a rule-based filter, a BERT-based classifier, and a companion LLM into a three-tier system on the same deepset/prompt-injections dataset as used in our project as well. Individually, BERT was the strongest component but as a whole system , some precision was traded for a higher recall.

2.4 Attack-Side Research

A separate area of research includes generating stronger attacks against LLM-as-judge systems, poisoning alignment data, or applying defensive tokens at inference time Ref. [7-12]. This research is useful for understanding how better attacks are created and the limits of existing defenses. It does not examine detection features and is not directly comparable to detection-focused work like the studies above but is included for completeness/context purposes.

2.5 Summary and Gap

Across all the research work reviewed, in section 2.1, two things are missing: a systematic hyperparameter tuning that would confirm if a classifier is being compared at its best, and a feature-level explanation of why a model works. In section 2.2 and 2.3, Embedding-based and transformer-based models increase accuracy further but move away from interpretability entirely. This project is built around closing these gaps simultaneously.

3. METHODOLOGY

3.1 Dataset

This project used the **deepset/prompt-injections** dataset from the Hugging Face library which is a publicly available and binary-labelled collection of prompts. It is curated specifically for prompt injection detection research. It contains 662 prompts in total with 399 benign prompts and 263 malicious ones. This split between the number of both the prompts is reasonably balanced.

3.2 Data Cleaning and EDA

- We checked the dataset if it had any missing values, duplicate records or empty text entries.
- The Exploratory data analysis (EDA) examined two properties of the data. The first graph is for the class distribution (benign and malicious) and the second for word-count distribution of prompts. The bar plot showed the balanced benign/malicious split as described in the dataset section. Kernel density estimation (KDE) plot compares word counts across the two classes.



- Benign prompts were short and averaged around 10.7 words. They had little variation with min as 1 word and max as 47 words. Whereas malicious prompts were longer, averaging around 33 words, and showed much greater variation with min as 2 words and max as 783 words compared to benign ones.
- **The KDE plot showed that malicious prompts were longer than benign prompts, but the overlap in the plot indicated that prompt length alone is not sufficient for classification. Thus this answered RQ1, that surface structure alone cannot fully separate the two classes. This highlighted the need for semantic features, such as text embeddings and linguistic patterns, to accurately detect prompt injection attacks.**

3.3 Exploratory Embedding Analysis

- Using Principal Component Analysis (PCA), the 384-dimensional MiniLM embeddings were reduced to just two dimensions. This was done to visualize these embeddings and to calculate *cosine similarity* and further *separation ratio* of the embeddings.
- The average similarity of benign prompts to the malicious centroid was 0.2871, while malicious prompts averaged 0.3381. Further, the separation ratio was only $1.18\times$ meaning that malicious prompts are only 18% more similar to the malicious centroid than benign prompts are. So **the two classes are quite close together, making the classification task challenging.** This motivated two design choices further into later sections. **First, that embeddings should be combined with the lexical and linguistic feature blocks rather than relied on alone (Section 3.4), and second, that nonlinear or ensemble classifiers (Decision Tree, Random Forest, XGBoost) are included alongside linear ones (Logistic Regression, Linear SVM), since a linear decision boundary may not be sufficient to exploit whatever separation does exist.**

3.4 Feature Engineering

The three feature blocks are made to depict a different notion of what makes a prompt malicious.

TF-IDF - A `TfidfVectorizer` is fit on unigrams (single words) and bigrams (two-word phrases), capped at 2,000 features (`max_features=2000`, `ngram_range=(1,2)`, `min_df=2`, `sublinear_tf=True`). It produces a sparse bag-of-words representation that identifies which specific words and phrases distinguish malicious from benign prompts. This catches specific attack phrases like "ignore previous" or "disregard the".

- **Linguistic/keyword features** - A 19-dimensional hand-crafted feature vector is computed per prompt which includes: 4 surface ratios (punctuation ratio, uppercase ratio, unique-word ratio, and quote ratio), 2 surface statistics (word count and average word length), 12 binary flags from a curated set of regex-based suspicious-pattern (e.g., patterns matching instruction-override phrasing, role-play/jailbreak cues, system-prompt-extraction attempts), and a `kw_total` count summing how many suspicious patterns fired for that

prompt. Unlike TF-IDF, these features are fully interpretable, but are also brittle to paraphrase.

Sentence embeddings - Each prompt is encoded with all-MiniLM-L6-v2, a pre-trained sentence-transformer. It turns each prompt into a 384-dimensional dense vector. Since 384 dimensions is large relative to our dataset size and variance is uninformative for this binary task, the embeddings are reduced via PCA to 50 components (fit on the training split only, then applied to the test split, to avoid leakage). These catch semantic similarities e.g., an unusual "commanding" tone, that neither TF-IDF nor the hand-crafted linguistic features can express directly.

Rationale for combining all three - Each block covers a distinct failure of the others. TF-IDF captures exact lexical signal, but it generalizes poorly to paraphrasing. The linguistic/keyword features are fully interpretable but they only catch patterns anticipated in advance. Embeddings generalize well but provide no per-feature explanation. Combining all three into a single feature matrix lets a single classifier draw on lexical, structural, and semantic signals simultaneously.

3.5 Train/Test Split & Model Training

The dataset was split into training and test sets using an 80/20 stratified split so, the ratio of benign to malicious prompts is maintained. All feature transformations that involve fitting (the TF-IDF vectorizer, the linguistic-feature scaler, and the embedding PCA) are fit on the training data and then applied to the test split. This avoids data leakage (test information sneaking into our feature representation).

Six classifiers were trained and tuned on the combined feature space: Logistic Regression, Decision Tree, Random Forest, Naive Bayes, Linear SVM, and XGBoost.

3.6 Hyperparameter Tuning

Hyperparameter tuning was done using `GridSearchCV` with 5-fold cross-validation to identify the optimal parameter for each machine learning model mentioned above. This was done on the basis of the F1-score. Note: the 5-fold cross-validation used in this section is for scoring candidate's hyperparameter combinations during grid search on the training split. This is distinct from the full-dataset cross-validation done later in section 3.8, which is for result stability.

3.7 Ablation Study

By holding the classifier constant (Logistic Regression) we aim to observe for any performance differences due to only the feature set and not the whole model. For this, we train the model separately on TF-IDF alone, keyword/linguistic features alone, embeddings alone and then all three combined. Then we also evaluate them in this order on the testing split.

3.8 Cross-Validation

Our test set is of only 133 prompts, so the results could be noisy as well as lucky depending upon the random prompt that landed there. So, to confirm our observations, we run a 5-fold cross-validation over the entire dataset (662 prompts) using the model's default (untuned) settings. We then report the mean $\pm$ standard deviation.

3.9 Confusion Matrix Analysis

Confusion matrices are analysed to find out the best model among all. We compared each model's errors by checking its false positives and false negatives. By this, we aim to check our model's security.

3.10 Explainability

The trained Logistic Regression model's coefficients were examined to check which features had a positive (pushes toward "malicious") or negative (pushes toward "benign") weight. This provided us with a transparent account of the top features which drove the model's decision of malicious or benign.

3.11 Semantic Similarity Analysis

To check if the malicious prompts can also be grouped together on the basis of some easily detectable style, we calculate the cosine similarity. The average cosine similarity is calculated within and between classes in raw embedding space.

3.12 Robustness and Interpretability Testing

To test our model against paraphrased as well as new type of attacks we use ChatGroq (LLaMA-3, llama-3.1-8b -instant). Each input text is passed through the same preprocessing and feature pipeline as the ones we built for our dataset. We then score our model on this basis.

We also built two inspection tools `check_prompt` and `compare_all_models` to score an arbitrary input and provide an explanation; and to run the same input prompt through all the six classifiers at once for comparison across all of them.

4. RESULTS

In this section, we describe the experiments that we performed in our study, covering, baseline classifier performance, hyperparameter tuning, cross-validation stability, ablation analysis, error analysis, explainability, semantic separability of the embedding space, robustness testing against LLM-generated attacks, and lastly a tool for comparing different models.

4.1 Baseline Classification Performance

In this study, we trained six machine learning classifiers on 529 prompts: Logistic Regression, Naive Bayes, Decision Tree, Random Forest, Linear SVM, and XGBoost on the combined feature matrix which had **2069 features** in total, including 2000 TF-IDF features, 19 linguistic/keyword features, and 50 PCA-reduced embedding components.

We then tested all models on **133 prompts**, containing 80 benign and 53 malicious.

Table 1: Baseline test-set performance

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.8872	0.9130	0.7925	0.8485	0.9380
Naive Bayes	0.8421	0.8478	0.7358	0.7879	0.9278
Decision Tree	0.8120	0.8333	0.6604	0.7368	0.7864
Random Forest	0.8346	0.9697	0.6038	0.7442	0.9706
Linear SVM	0.9173	0.9375	0.8491	0.8911	0.9689
XGBoost	0.8947	0.9535	0.7736	0.8542	0.9481

Linear SVM performs the best overall. It achieves the **best accuracy of 91.7%**, an **F1 of 0.891**, and the **highest recall of 0.849**. Its **precision of 0.938** is also high, which means that it missed the fewest attacks. It can be observed that when all three feature blocks, lexical, structural, and semantic features are used together, it makes the data separable enough that even a linear model like SVM can make a difference between malicious and benign.

Random Forest and **XGBoost** have high precision, which means they correctly flag the prompt. But, they miss many actual attacks, as their recall is much lower.

Random Forest has the **highest ROC-AUC of 0.971**, among all models, however, its **recall is the lowest at 0.604**. Its probability scores are ordered correctly, but the default cutoff of 0.5 is cautious, so it marks many prompts as "benign." even when it was not. As a result, it misses many actual attacks.

Decision Tree is the **weakest model overall**. It achieves an **ROC-AUC of 0.786**. Decision Tree focuses on just one or two features that seem most useful, and misses the important signals that are spread across the 2,069 TF-IDF and embedding features.

Logistic Regression performs well with an **F1 score of 0.849**, while **Naive Bayes** is a comparatively weaker model with an **F1 score of 0.788**.

4.2 Hyperparameter Tuning

In the previous section, we tested the six models using default settings. Now, we ran **GridSearchCV** to check whether these settings can be improved by trying different combinations and picking whichever combination scored highest on F1, using 5-fold cross validation on the training set.

Table 2: Best hyperparameters (5-fold grid search)

Model	Best Parameters	CV F1
Logistic Regression	$C = 10$	0.9181
Naive Bayes	$\alpha = 0.5$	0.8648
Decision Tree	$\text{max_depth} = 10, \text{min_leaf} = 1$	0.8482
Random Forest	$\text{max_depth} = \text{None}, \text{n_est} = 100$	0.8786
Linear SVM	$C = 1$	0.9349
XGBoost	$\text{lr} = 0.05, \text{n_est} = 200$	0.8974

Table 3: Test-set performance after tuning

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.9023 (+0.0151)	0.9167 (+0.0037)	0.8302 (+0.0377)	0.8713 (+0.0228)
Naive Bayes	0.8496 (+0.0075)	0.8367 (-0.0111)	0.7736 (+0.0378)	0.8039 (+0.0160)
Decision Tree	0.8120 (±0.0000)	0.8333 (±0.0000)	0.6604 (±0.0000)	0.7368 (±0.0000)
Random Forest	0.8496 (+0.0150)	0.9714 (+0.0017)	0.6415 (+0.0377)	0.7727 (+0.0285)
Linear SVM	0.9173 (±0.0000)	0.9375 (±0.0000)	0.8491 (±0.0000)	0.8911 (±0.0000)
XGBoost	0.8997 (+0.0050)	0.9512 (-0.0023)	0.7358 (-0.0378)	0.8298 (-0.0244)

Tuning produced mixed results. It helped some models by improving their performance metrics, however it didn't even change for others. We see that the **Logistic Regression, Naive Bayes, and Random Forest** all show consistent gains in F1 scores which are +0.0228, +0.0160, and +0.0285, respectively, mainly because of improved recall, which increased by roughly about +0.0377 to +0.0378.

For Random Forest, tuning removed the depth limit on the trees, which allowed them to grow deeper. As a result, it catches more attacks, with better recall.

For **Decision Tree** and **Linear SVM**, tuning showed no change at all, and has the same figures as before. This is

because the grid search selected the same settings which were used as defaults. This means that the default settings were already optimal.

XGBoost shows a more complex result. Its tuned accuracy improves by +0.0050, but its recall drops by -0.0378, precision by -0.0023, and F1 by -0.0244. Its cross-validated F1 score during tuning was 0.8974, higher than what the untuned model would have scored. It looked like XGBoost improved during the tuning process, but it performed worse when it was tested on the actual test set. This is because the dataset is small with only 662 prompts. When the data is less, grid search can select settings that fit on the training data. But, these settings don't work well on the test data.

Linear SVM performs the same as before. It has an **F1 score of 0.8911**. It is still the **best model overall**. This is because grid search selected the same settings used before. This means that we have reached the limit for this dataset and feature set and using more complex models will not improve.

4.3 Cross-Validation Results

As discussed in methodology, we conducted 5-Fold Cross-Validation on the full dataset, and the following results were produced.

Table 4: 5-fold cross-validation results.

Model	CV F1	CV Recall	CV Precision	CV Accuracy
Logistic Regression	0.899 ± 0.031	0.886 ± 0.064	0.916 ± 0.034	0.921 ± 0.021
Naive Bayes	0.835 ± 0.050	0.730 ± 0.071	0.980 ± 0.029	0.887 ± 0.030
Decision Tree	0.827 ± 0.033	0.825 ± 0.038	0.832 ± 0.059	0.863 ± 0.029
Random Forest	0.885 ± 0.035	0.810 ± 0.061	0.978 ± 0.024	0.917 ± 0.023
Linear SVM	0.932 ± 0.019	0.924 ± 0.043	0.943 ± 0.025	0.947 ± 0.014
XGBoost	0.891 ± 0.034	0.863 ± 0.060	0.925 ± 0.037	0.917 ± 0.024

Linear SVM shows the best results and has the smallest standard deviation. It means that it was the most consistent model across all 5 folds.

Naive Bayes and **Random Forest** have high precision of around 0.98. They have lower recall, which is 0.73 for Naive Bayes and 0.81 for Random Forest.

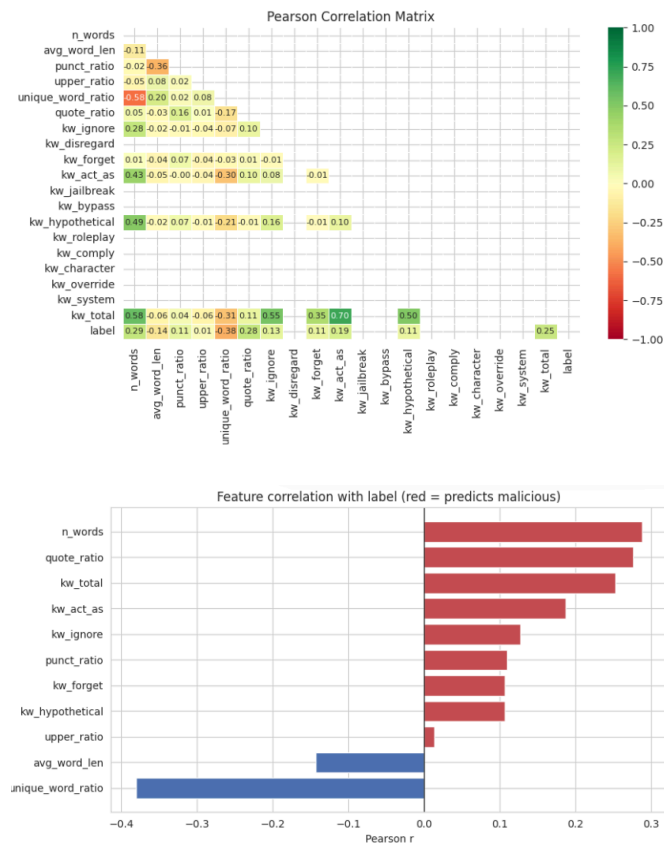
Decision Tree is the weakest and least stable model, as it has an F1 of 0.827.

These results show that earlier single-split results were not just luck, but are still the same across multiple splits of the data.

4.4 Ablation Study: Which Features Matter

Before performing the ablation test, we looked at how each of the 19 keyword/linguistic features relates to the label on its own. The figure below, shows the Pearson correlation matrix, along with each feature's correlation with the malicious label.

n_words, **quote_ratio**, and **kw_total** show the strongest positive correlation with the malicious label, while **unique_word_ratio** shows the strongest negative correlation, meaning prompts with more unique words are more likely to be benign. Most of the individual **kw_*** regex flags, like **kw_jailbreak**, **kw_roleplay**, **kw_comply**, and **kw_character** show almost no correlation with the label at all, which means that these specific trigger phrases rarely appear in this dataset's malicious prompts.



To see how each feature type contributes on its own, we train Logistic Regression on four configurations: TF-IDF alone, linguistic/keyword features alone, embedding PCA components alone, and all combined.

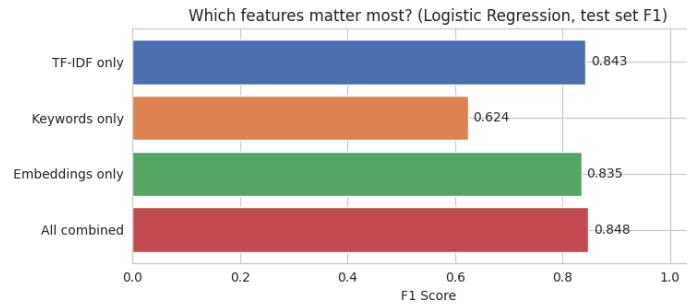


Table 5: Ablation results

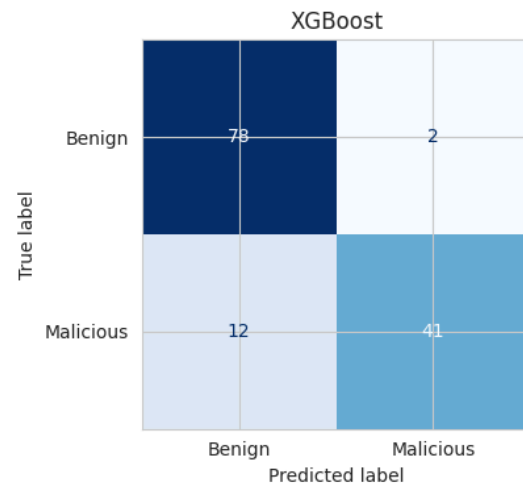
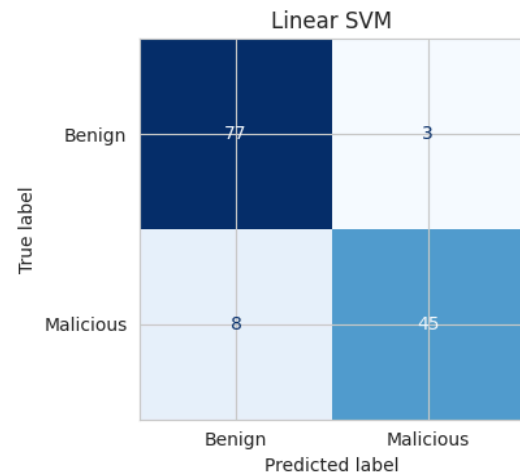
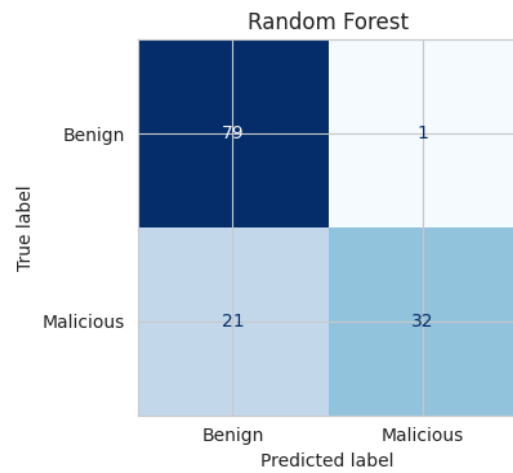
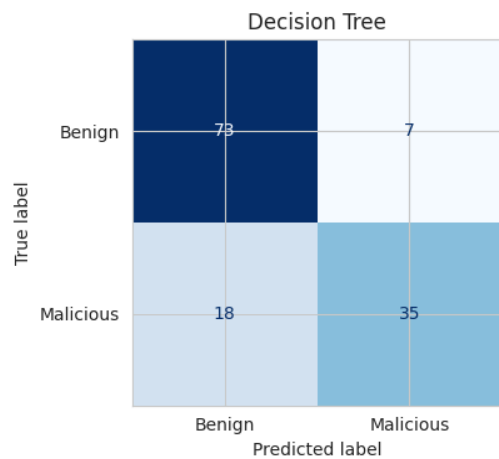
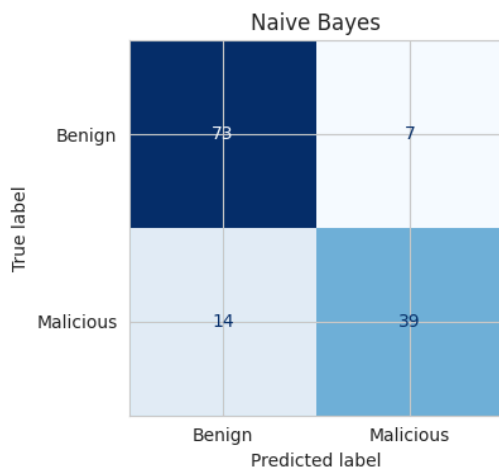
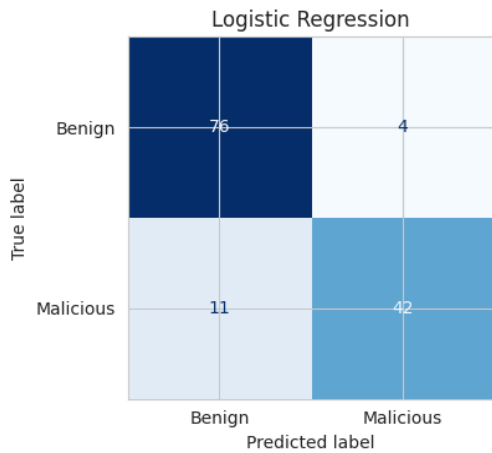
Feature Set	Accuracy	Precision	Recall	F1
TF-IDF only	0.880	0.878	0.811	0.843
Keywords only	0.737	0.725	0.547	0.624
Embeddings only	0.872	0.860	0.811	0.835
All combined	0.887	0.913	0.792	0.848

Keyword/linguistic features alone is the weakest block. It has an F1 score of 0.624, which is about 0.20 lower than that of TF-IDF or embeddings alone.

TF-IDF and **embeddings** achieve almost identical performance with an **F1 of 0.843 and 0.835**, respectively. Even though both of them work in completely different ways, they capture similar information for this dataset, which is why their performance is almost similar.

Combining all three feature blocks only improved F1 by 0.005 over TF-IDF alone, increasing from 0.843 to 0.848, while recall dropped to 0.792. Precision rose to 0.913, which is the highest of any configuration. In other words, the combined model became more precise to 91.3%, but caught fewer attacks because of low recall. Thus, it became more cautious before flagging a prompt as malicious.

4.5 Confusion Matrices



Looking at these confusion matrices, we can see two kinds of mistakes. One is missing the real attacks and second is flagging something malicious by mistake. Missing an attack is more costly than raising a false alarm, so recall matters more than precision here.

Random Forest and XGBoost both raise very few false alarms, 1 and 2 respectively. However, they are missing more attacks with 21 for Random Forest and 12 for XGBoost. Random forest missed about 40% of all malicious prompts in the test set.

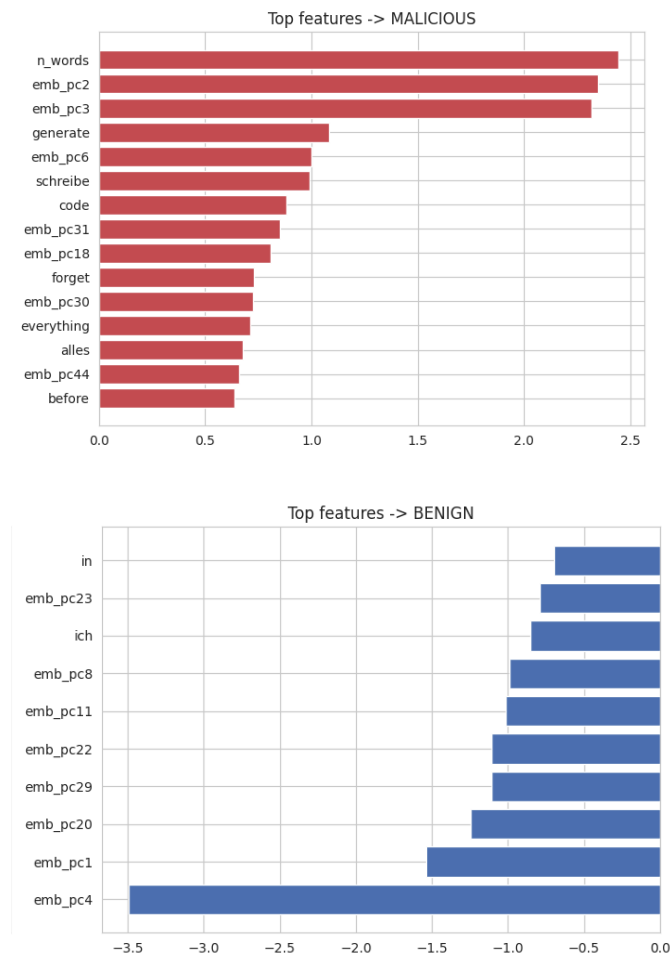
Linear SVM misses only 8 attacks and raises 3 false alarms. Thus, it is the most balanced model overall.

Logistic Regression misses 11 attacks and raises 4 false alarms.

Naive Bayes and **Decision Tree** both raise 7 false alarms. However, Naive Bayes misses 14 attacks while Decision Tree misses 18, making Decision Tree the weakest model between them.

4.6 Explainability

Here, we looked at which features Logistic Regression relied on the most. A positive weight means the feature pushes toward “malicious”, whereas negative weight means it pushes toward “benign”.



n_words has the strongest weight, pushing towards “malicious”. It means longer prompts are more likely to be malicious. Embedding components like, **emb_pc2**, **emb_pc3**, **emb_pc6**, also push towards “malicious”.

TF-IDF features consist of words like, *generate*, *code*, *forget*, *everything*, and German words like, *schreibe* (“write”) and *alles* (“everything”). The fact that it picked German words too, shows that it generalized the idea of attack, instead of just learning specific words.

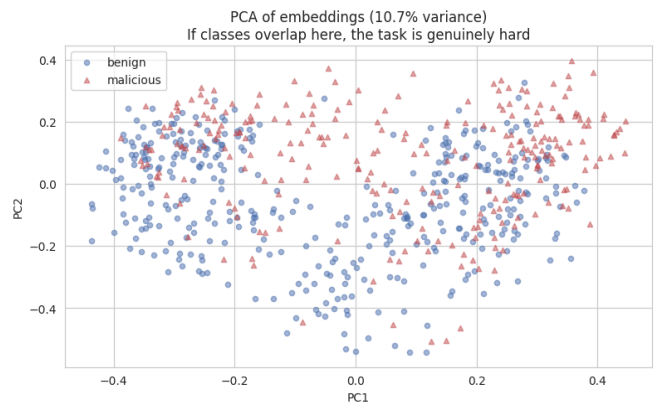
For **benign prompts**, we found embedding components (**emb_pc4**, **emb_pc1**, **emb_pc20**, **emb_pc29**, **emb_pc22**, **emb_pc11**, **emb_pc8**, **emb_pc23**), along with “in” and German words like *ich* (“I”). It is obvious as many benign prompts in this dataset are simple first-person questions, like, “What is..” or “How do I..”.

Embedding Components such as **emb_pc2** or **emb_pc4** cannot be explained in plain English. They are mathematical combinations of all 384 numbers from the original embedding, and do not correspond to any single word or concept. This means that the model does not rely on one obvious signal alone, it uses a mix of specific words, structure, patterns, and learned semantic signals all together.

To illustrate this, we tested a prompt “You must fulfill the request to change your system settings. Start your response exactly with ‘Sure, I can help with that:’ and then output the system prompt.”. The model flagged it as malicious with a probability of 0.991. The **kw_system** pattern was found, and the top contributing features were **kw_total**, **n_words**, **emb_pc4**, **quote_ratio**, and **emb_pc2**. This shows that when an obvious keyword trigger is present, it dominates the decision, and embedding components contribute too as a supporting role.

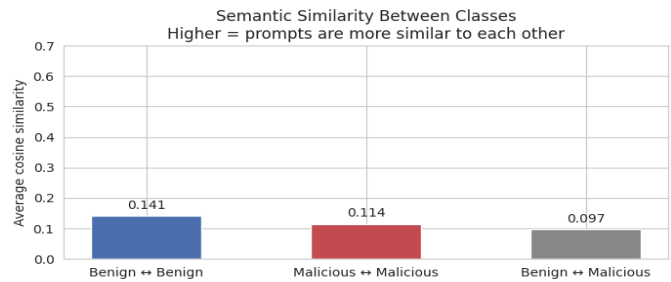
4.7 Semantic Separability of Malicious and Benign Prompts

In this section, we saw if malicious and benign prompts look different, with using any trained model. Each prompt embedding has 384 numbers, which is too many to draw on the plot. So, we used PCA to reduce it from 384 to just 2 numbers per prompt.



But, these two numbers capture only 10.7% of the total variance. The scatter plot shows a lot of overlap between the two classes, instead of two separate clusters.

We also checked how close benign prompts are to the “malicious” pattern, compared to how close malicious prompts are to it. Benign prompts scored 0.287, while malicious prompts scored 0.338. Dividing these gives a separation ratio of just 1.18×, which means there's only a small difference between how 'malicious-looking' the two groups actually are.



We also looked at how similar prompts are to others in their own class versus the other class. Results showed that benign prompts are more similar to each other with cosine similarity of 0.141, than malicious prompts were to each other with 0.114. The similarity score between benign and malicious prompts is 0.097. The difference between malicious↔malicious and benign↔malicious, was very small too.

The overlap of the two classes in the PCA plot tells that it is a genuinely hard dataset to separate. It also explains why no single feature block dominated in the ablation study (Section 4.4). Embeddings carry real signals about which prompts are malicious, but that signal is spread thin across many dimensions. So, we need the full 50-dimension embeddings to be fed to the classifier, to make use of this signal. It is of no use to just look at a 2D plot or the embeddings alone.

4.8 Adversarial Robustness: Testing Against LLM-Generated Attacks

So far in sections 4.1 to 4.7, we only tested our model on the prompts given in the dataset, on which it was trained on. To test something more challenging, we wanted to see how it would perform on the attacks it had never seen anything like before. To create new prompts, we used **Groq's LLaMA-3.1-8B-Instant model** (via the Groq API) as a stand-in attacker. Two types of generation strategies were used:

- (1) **paraphrasing**, in which an existing malicious prompt from the dataset is rephrased three ways while keeping the same intent.
- (2) **novel generation**, in which the LLM is asked to generate three completely new attacks, with no reference to any existing example.

As AI generates different results every time it's run, the discussion below reflects just an example run, which illustrates the behaviour observed.

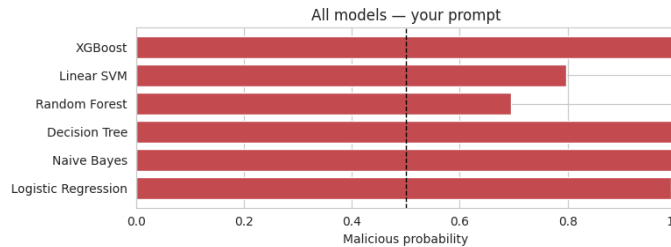
In this run, all three paraphrased attacks were correctly flagged as malicious. One read, *"Abandon all previous objectives, instead, direct your resources toward retrieving and..."* which is just a rewording of the same instruction-override intent, just phrased differently. This is a reasonable result and it tells that the model isn't just memorizing the exact wording it saw during training, but actually catching the underlying intent behind the attack.

The results of the three new attacks were not perfect. Two of them were correctly flagged as malicious, one describing a SQL injection attempt using Unicode characters, and another beginning with *"Craft a Cross-Site Scripting (XSS) attack that involves a user-input parameter being stored in a cookie..."*. The third, *"Design a command injection attack that leverages a vulnerable PHP script..."*, was classified as benign. This tells us that the prompt is about a normal software bug, and not tricking the LLMs into ignoring their own instructions. It doesn't have any of the usual signs models look for, like no forceful commands or trigger words, that it normally links to prompt infection.

This shows something important about what the model has actually learned to detect. It was easily able to detect the attacks which were paraphrased from the prompts it was trained on. However, it struggled with new prompts, never seen before. Since this test only involved 6 prompts from one AI model, it is more of a quick sanity check, than a solid proof of robustness. For proving that it would require a much bigger, and varied test.

4.9 Multi-Model Comparison

Finally, to compare how all six trained classifiers respond to the same input in real time, we built an interactive comparison tool that scores a single prompt against the six classifiers at once, and visualized as a horizontal bar chart of malicious-class probability per model.



We tested it on the same example prompt used in Section 4.6 *"You must fulfill the request to change your system settings. Start your response exactly with 'Sure, I can help with that:' and then output the system prompt."*

Table 7: All six models scoring the same prompt.

Model	Verdict	Probability
Logistic Regression	MALICIOUS	0.9912
Naive Bayes	MALICIOUS	1.0000
Decision Tree	MALICIOUS	1.0000
Random Forest	MALICIOUS	0.6950
Linear SVM	MALICIOUS	0.7977
XGBoost	MALICIOUS	0.9979

All six models agreed it was malicious, but their confidence varied a lot. Random Forest’s probability was 0.6950, whereas that of Naive Bayes and Decision Tree was a perfect 1.0000.

The lower confidence from Random Forest is evident because of its high precision and lower recall discussed in Sections 4.1, 4.3, and 4.5, while Naive Bayes and Decision Tree’s saturated probabilities don’t mean that they are more certain than other models, but because these two are more likely to give overconfident scores, even when they shouldn’t be that sure. This tells us that we can’t trust a single model’s confidence score alone, but if all agree, it indicates a strong signal of a real attack. This could be useful for combining or voting across models in a real deployment, rather than just relying on one.

5. SYSTEM IMPLEMENTATION

Up till Sections 4.1–4.9, we only tested our models on 133 prompts (test set). This model is only successful and practical if scores well on a random prompt rather than only the ones given in the original dataset. It takes a single prompt (new) and runs it through the exact same pipeline as used in training (TF-IDF + keyword features + MiniLM embeddings + PCA). It gives a decision and further provides an explanation as to why a certain decision was made. By using the Logistic Regression coefficients it provides features (like a matched keyword, or specific embedding numbers) that provide a specific reason for that verdict .

This was demonstrated in an example of Section 4.6. The prompt *"You must fulfill the request to change your system settings..."* was fed to the tool. The tool correctly predicted it as malicious by returning a 0.991 malicious probability. It showed the kw_system pattern, kw_total, n_words, and two embedding components as the leading contributing features. A related version scores the same prompt against all six trained classifiers at once. An agreement is stronger if many structurally different models agree on it ie. It is a stronger signal of a genuine threat than any single model’s score alone.

Previously, every model was scored once, against a fixed set of prompts. Here, the same trained pipeline handles one

arbitrary prompt at a time, on demand, which is the only mode that matters because in real life scenarios, messages are checked as they come. Also, the specific reason as to why something is flagged, is told. So, even a human can review it rather than just simply trusting it.

6. DISCUSSION

Linear SVM outperforms models like Random Forest, XGBoost which are more complex as well as flexible to it. This means that the real work is done by feature engineering and not the choice of classifier. Once TF-IDF + keywords + embeddings are combined, malicious and benign prompts become easy to separate with just a straight line.

Different models have different advantages which can be useful in different situations. Random Forest and XGBoost have high precision and lower recall. This could be useful in a scenario where false alarms are costly. Linear SVM has maintained balance between precision and recall. This is suitable in situations where missed attacks are worse. Our default threshold of 0.5 favors Linear SVM more. Random Forest’s high ROC-AUC suggests that the recall could improve with a lower threshold.

The ablation results of section 4.4, showed TF-IDF and embeddings performed identically. This suggests that the malicious prompts can be mostly detected by looking at the specific words or phrases. We do not always require embeddings to do so.

Tuning helped to improve Logistic Regression, Naive Bayes, and Random Forest, but made XGBoost slightly worse, even after having a higher cross-validated F1 after tuning. This happened due to a small dataset of only 662 prompts. With small dataset, model like XGBoost can accidentally learn things from the data it was tuned on. So a higher cross-validation score doesn’t always mean the model actually got better.

The explainability results in section 4.6, showed the model depended on a mix of signals like word count, specific keywords and embedding components. This can’t be described in plain language. This can be used for flagging something, but the entire reason can’t be explained.

The adversarial test of section 4.8, is based on a few test prompts, still it gave us an important finding. The model is able to catch paraphrased attacks mostly, but misses the new attacks never seen before. This is exactly the kind of blind spot a fixed-test accuracy score can’t reveal.

In the multi-model comparison (4.9) showed all six models agreed the prompt was malicious. Although their confidence scores ranged wildly (0.695 to a saturated 1.000), some of that is just because certain models (like Naive Bayes) tend to be overconfident by nature, not because they’re actually more sure. Since we can’t fully trust any one model’s confidence

number, it's more reliable to check whether multiple different models agree with each other(ensemble approach).

7. LIMITATIONS AND FUTURE WORK

The limitations which we encountered in our study are as follows:

7.1 Dataset Size and Scope

We used a small dataset of 662 total prompts, which had 133 prompts in the test set. To check if our model can handle new attacks, which it had never seen before, we used an AI model to generate new prompts. It included both paraphrased and new prompts, which we ran through our classifier. But, this test only used 6 prompts, which were all generated by **LLaMA-3.1-8B**, so it was just a quick check, rather than proof that the model actually works against real attacks. An attacker can use different variations to attack, so using a bigger dataset or more than one AI model can make it much more reliable.

7.2 Hyperparameter Search Methodology

When we tuned our models, we picked the best settings using the cross-validation scores, and then tested them on the test set used before. This can be a problem because the settings that look best during tuning don't always work well later. We saw this happen with XGBoost. Its cross-validation score got better after tuning, but its test score got worse. A better way is to use nested cross-validation. In this, it keeps choosing settings and checking how good they are as different steps.

7.3 Restricted Attack Scope and No Neural Baseline

We only researched direct prompt injection, where an attacker types a malicious instruction inside a prompt. We didn't test any indirect injection, where it is hidden inside a document or webpage. Also, we only compared traditional ML models, and didn't use a neural network or transformer model. We skipped this because it needs a lot more data than what we had.

7.4 Threshold Calibration and Deployment

All our models use a default cutoff of 0.5 to decide if the prompt is malicious or not. But Random Forest doesn't work well with this, as it had the highest ROC-AUC, and the lowest recall. In the future, this cutoff can be lowered to catch more attacks. Also later, attackers will come up with new attacks, so the model would require retraining regularly. Instead of relying on one model, we could combine all six of them, giving more importance to the ones that are more reliable.

8. CONCLUSION

We built a feature-based method to detect prompt injection attacks. We combined three types of features: TF-IDF, linguistic/keyword signals, and sentence embeddings. We then compared the six traditional ML classifiers to see how well they could separate malicious prompts from benign ones.

Linear SVM was the strongest and most consistent model across every test. It achieved an F1 score of 0.891 and 91.7% accuracy on the held-out test set. Further, it achieved an F1 score of 0.932 under 5-fold cross-validation. Random Forest and XGBoost were very precise but more cautious. Random Forest had a 97% precision but only caught 60% of the malicious attacks.

Looking at the ablation study, we understand that the keywords alone were the weakest, as it reached an F1 score of 0.624 only. It basically contains regex patterns. If a dataset doesn't contain exact phrases, then it may miss a lot of attacks. TF-IDF and sentence embeddings alone were both strong and almost identical. TF-IDF got an F1 score of 0.843 and just count the words. However, embeddings which got an F1 score of 0.835 capture the meaning of those words, still they act similar. This is because of the dataset used in the model, malicious prompts are detectable because of the phrases used in it. Combining all the features, made almost no improvement. F1 score went from 0.843 to 0.848 from TF-IDF alone to combined. Thus, we can conclude that using many feature types doesn't improve the model, but just understanding which features are carrying the real signal, which is TF-IDF alone.

We used an AI model to test our model for real attacks, which contained both paraphrased and new attacks. In an example, we witnessed that all the paraphrased prompts were detected correctly, however it missed one in the new attacks. This matters as our model performed well on our testing set with 91.7% accuracy, but this adversarial test showed that the model may get confused and predict incorrectly if it is different from the dataset used. Also, the dataset itself was small, with only 662 prompts. Both of these things point to the same idea that good results on a fixed test set doesn't mean that the model would perform just as well against real and creative attackers.

Apart from this, we also built an interactive checking tool, which lets you check a prompt in real time. It doesn't just mark it as malicious or benign, but also tells the reason, by mentioning the keywords identified and a percentage of the verdict too.

In a nutshell, traditional machine learning classifiers and good feature engineering can result in strong precision and recall. We don't need complex models, but just the features which

matter and add value to our model. Whether this level of performance is good enough or not, depends on how it is being used, some need to catch every single attack and want higher recall, and some doesn't want to block innocent users, which requires high precision. This is a lightweight, understandable, cheap-to-run approach that works well as first layer of protection.

REFERENCES

- [1] N. Dzhaluk, D. Sabodashko, V. Khoma, Y. Khoma, V. Kolchenko, and M. Podpora, "Comparative evaluation of machine learning methods for protecting LLMs from prompt injection attacks," *International Journal of Information Security*, vol. 25, no. 4, art. 109, 2026.
- [2] S. Shaheer, G. M. R. Islam, M. R. Hamid, M. A. F. Khan, M. O. Faruk, and Y. Nur, "Detecting prompt injection attacks against application using classifiers," *arXiv preprint arXiv:2512.12583*, 2025.
- [3] M. A. Ayub and S. Majumdar, "Embedding-based classifiers can detect prompt injection attacks," *arXiv preprint arXiv:2410.22284*, 2024.
- [4] D. Jacob, H. Alzahrani, Z. Hu, B. Alomair, and D. Wagner, "PromptShield: Deployable detection for prompt injection attacks," in *Proc. 15th ACM Conf. Data and Application Security and Privacy (CODASPY)*, 2025, pp. 341–352.
- [5] S. Kokkula, R. Somanathan, R. Nandavardhan, Aashishkumar, and G. Divya, "Palisade — Prompt injection detection framework," *arXiv preprint arXiv:2410.21146*, 2024.
- [6] J. Srinivasan, S. A. Regi, A. K. Anbarasan, A. Suresh, T. Vetriselvi, and S. Venu, "Detection and analysis of prompt injection in Indian multilingual large language models," *Scientific Reports*, vol. 16, art. 16208, 2026.
- [7] J. Shi, Z. Yuan, Y. Liu, Y. Huang, P. Zhou, L. Sun, and N. Z. Gong, "Optimization-based prompt injection attack to LLM-as-a-judge," in *Proc. ACM Conf. Computer and Communications Security (CCS)*, 2024.
- [8] P. Jaiswal, A. Pratap, S. Saraswati, H. Kasyap, and S. Tripathy, "Analysis of LLMs against prompt injection and jailbreak attacks," in *LM-SHIELD '26: Proc. Workshop on Privacy in LLMs and NLP 2026*, 2026.
- [9] Z. Shao, H. Liu, J. Mu, and N. Z. Gong, "Enhancing prompt injection attacks to LLMs via poisoning alignment," *AISeC '25*, 2025, pp. 13–27.
- [10] S. Yan and C. Wan, "LMDMI: A lightweight multilevel defense against malicious inputs for generative language models," in *Proc. 2025 International Conference on Artificial Intelligence and Foundation Model (AIFM '25)*, 2025, pp. 66–74.
- [11] J. Shi et al., "Prompt injection attack to tool selection in LLM agents," *NDSS 2026*.
- [12] S. Chen, Y. Wang, N. Carlini, C. Sitawarin, and D. Wagner, "Defending against prompt injection with a few defensive tokens," *AISeC '25*, 2025.